Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 13

Recap and outline

- Feature selection - what we've covered
- Going forward - applications and connections
- Today: introducing graphical model selection

Feature selection overview

- Mathematical definition
- Methods/algorithms (filter, embedded, wrapper)
- Validation/evaluation

Mathematical definition(s)

Motivating risk-based criteria:

$$\widehat{S^*} \in \arg \min_{S \subseteq \{1,2,\ldots,p\}, |S| \leq s} \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(Y^{(i)}, f(X_S^{(i)}))$$

Conditional independence perspective:

$$S^* \in \arg \min_{S \subseteq \{1,2,\ldots,p\}, Y \perp X_{S^c} | X_S} |S|$$

Feature-wise conditional independence definition:

$$\begin{aligned} Y &\perp X_j | X_{-j}, \quad \text{for all} \quad j \in \tilde{S}^c \\ Y &\not\perp X_j | X_{-j}, \quad \text{for all} \quad j \in \tilde{S} \end{aligned}$$

Methods/algorithms

- Filter: correlation, mutual information, conditional correlation
  - Independent of prediction function class /algorithm $\mathcal{F}$
  - Lose information about class $\mathcal{F}$
- Embedded: OLS+p-values, Lasso, MDI
  - Do not need to optimize over subsets as it comes free with prediction
  - Only applies to scenarios where parameters associated with prediction function contain feature selection information
- Wrapper: LOCO, backward selection, forward selection (AIC or BIC)
  - Need to optimize over both leading to significant computational challenges
  - Applies to any function class $\mathcal{F}$

Validation/evaluation methods

- Out-of-sample prediction error
  - Default and quantitative
  - May over-select features
- Domain knowledge
  - Often subjective and not always straightforward to find
  - Can be biased by domain knowledge
- Stability
  - We can make it quantitative
  - Can apply to both stability of algorithms and stability of features under data perturbations
- Simplicity/interpretability
  - Again subjective and context-dependent

Going forward

Connecting feature selection to:

- Feature selection and graphical models
- Interpretable machine learning (feature attribution)
- Hidden/learned features

Feature selection and conditional independence

Conditional independence perspective of feature selection:

$$S^* \in \arg \min_{S \subseteq \{1,2,\ldots,p\}, Y \perp X_{S^c} | X_S} |S|$$

Feature-wise conditional independence definition:

$$\begin{aligned} Y &\perp X_j | X_{-j}, \quad \text{for all} \quad j \in \tilde{S^c} \\ Y &\not\perp X_j | X_{-j}, \quad \text{for all} \quad j \in \tilde{S} \end{aligned}$$

- The conditional independence perspective naturally connects to graphical models
- As we see later, learning a graphical model from data is equivalent to performing feature selection at each node through this conditional independence relationship

Connection to graphical models

- Graphical models visual representation of conditional independence relationships
- Often unsupervised $(X_1, X_2, ..., X_p)$
- Each node is feature X_k
- Each edge/non-edge relationships a conditional dependence/independence relationship
- As we show later, selecting *neighbors* of node X_k is equivalent to feature selection

What are graphical models?

- Consider nodes/feature $(X_1, X_2, ..., X_p)$ (Y could also potentially be a node)
- Combines probability/statistics and graph theory
- Edges/non-edges between nodes reflect conditional dependence/independence relationships
- Graph: $G = (V, E)$ where $V = (X_1, X_2, ..., X_p)$, E is edge set
- Graphs widely used in algorithms/other contexts
- Visual representation of conditional dependence/independence relationships of multi-variate joint distributions
- Directed edges potentially also encode causal/directional relationships
- Examples: time series, spatial/image data, e.t.c.

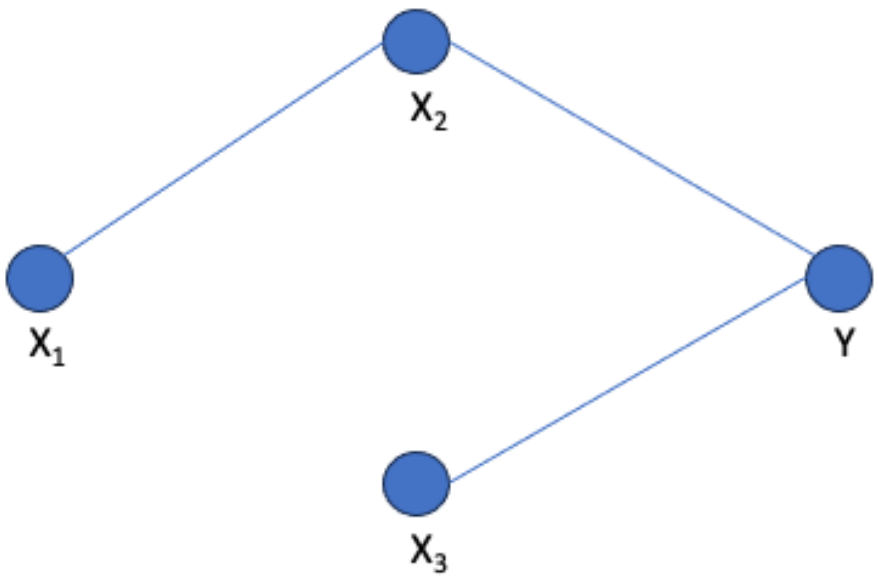
Why are they interesting/well-studied?

- Visual representation is highly interpretable
- Naturally amenable to many applications (e.g. temporal relationships, spatial relationships, image/language data, e.t.c.)
- Can represent causal/directional relationship
- Simple way to explain seemingly confusing behavior

Recall: Housing data example

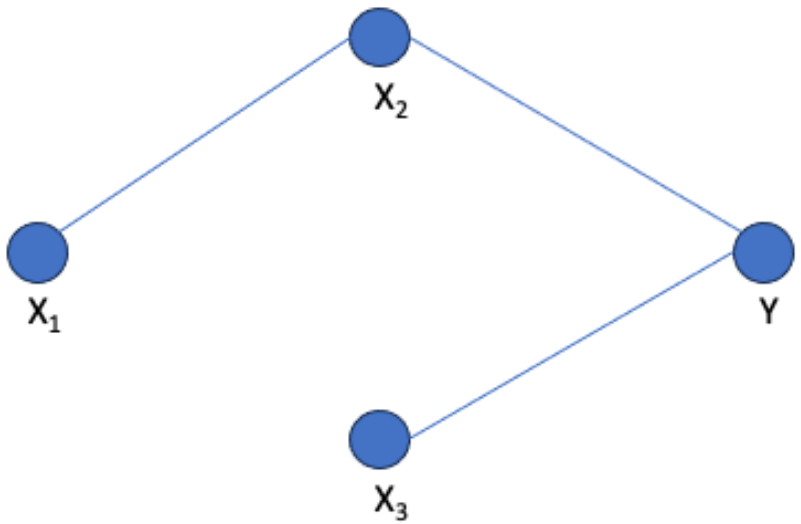
House prices with $p = 3$ features: square footage, number of bedrooms, and a neighborhood quality score. Y : House price; X_1 : Square footage; X_2 : Number of bedrooms; X_3 : Neighborhood quality score. Below is a snapshot of the data.

$$Y \not\perp X_1, \quad Y \perp X_1 | X_2$$



Example: Simpson's paradox

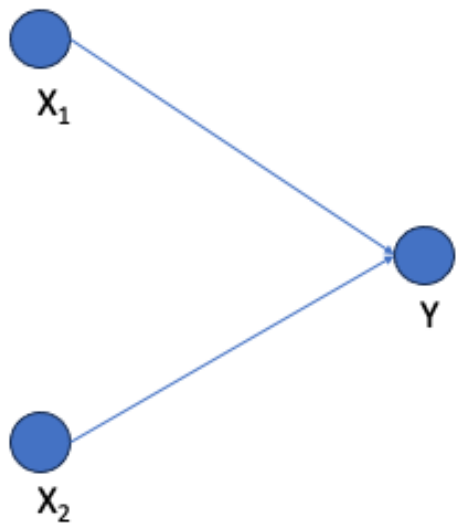
- Simpson's paradox commonly raised in probability/statistics
- Simply refers to the fact that conditioning on another feature(s) changes the nature of the nature of the relationship
- I.e. dependence to independence or positive to negative correlation
- Not actually a paradox and straightforward to represent with a graphical model
- E.g. $Y \not\perp X_1, \quad Y \perp X_1 | X_2$



Example

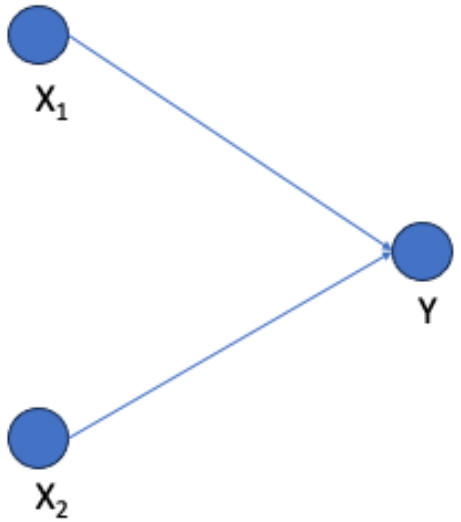
Restaurant with $p = 2$ features: burger quality and fries quality. Y : Does restaurant survive 6 months; X_1 : Quality of burgers; X_2 : Quality of fries. Below is a snapshot of the data.

$$Y = \begin{cases} 1, & \text{if } X_1 + X_2 \geq 1 \\ 0, & \text{otherwise} \end{cases}$$



Example: Berkson's paradox

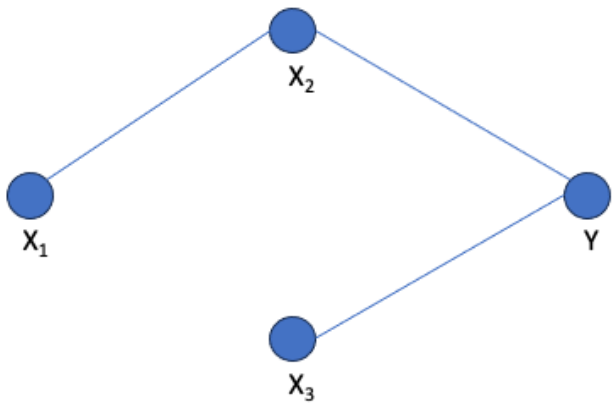
- Survival/selection bias
- Conditioning on response can lead to dependence between features
- $X_1 \perp X_2, X_1 \not\perp X_2 | Y$
- Again not a paradox just simply how conditioning can induce a relationship
- Represented by a directed graphical model (directed edges)
- Reflects issues with conditioning on response



Connecting graph to conditional independence (undirected)

Undirected graph example: $Y \perp X_1 | X_2$, $X_1 \not\perp X_2$, $Y \not\perp X_2 | (X_1, X_3)$, $Y \not\perp X_3 | (X_1, X_2)$

$$E = \{(X_1, X_2), (X_2, Y), (X_3, Y)\}$$

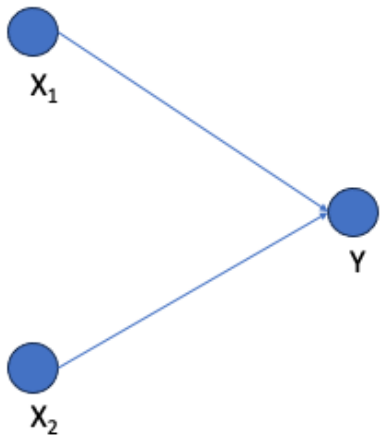


Non-edge/edge represents a conditional independence/dependence relationship

Connecting graph to conditional independence (directed)

Directed graph example: $X_1 \perp X_2, Y \not\perp X_1, Y \not\perp X_2, X_1 \not\perp X_2|Y$

$$E = \{(X_1, Y), (X_2, Y)\}$$



Non-edge/edge represents a conditional independence/dependence relationship

Two kinds of graphical models

- Undirected graphical models (edges undirected)
- Directed graphical models (edges directed)
- Undirected graphical models represent association/conditional dependence relationships (multi-variate distributions)
- Directed graphical models represent directional/causal/structural relationships (structural equation models)

Undirected graphical models

Undirected graphical models:

$$G = (V, E)$$

where $V = (X_1, X_2, .., X_p)$, E undirected edges
Nice and simple relationship:

$$(X_j, X_k) \in E \Leftrightarrow X_j \not\perp X_k | X_S$$

where $S = \{1, 2, ..., p\}/\{j, k\}$
In words: There exists an edge $(X_j, X_k) \in E$ if and only if you condition on all other nodes/random variables, X_j and X_k are dependent

Undirected graphical models and conditional independence

For undirected graphical models:

$$(X_j, X_k) \in E \Leftrightarrow X_j \not\perp X_k | X_S$$

or alternatively

$$(X_j, X_k) \notin E \Leftrightarrow X_j \perp X_k | X_S$$

where $S = \{1, 2, ..., p\}/\{j, k\}$
Shows how undirected graphical models connect to conditional independence but how does it connect to feature selection?

Undirected graphical models and feature selection

For feature selection:

$$X_j \in \tilde{S} \Leftrightarrow Y \not\perp X_j | X_{-j}$$

For undirected graphical models:

$$(X_k, X_\ell) \in E \Leftrightarrow X_k \not\perp X_\ell | X_S$$

where $S = \{1, 2, ..., p\}/\{k, \ell\}$.
Question: Is there a connection between whether there is a feature X_j in $\tilde{S}$ or there is an edge between $(X_k, X_\ell) \in E$?

Neighborhood selection as feature selection

Define a *neighborhood set* for node X_k:

$$\mathcal{N}(X_k) = \{\ell : \ell \subset \{1, 2, ..., p\}/\{k\}, (X_k, X_\ell) \in E\}$$

Recall that:

$$(X_k, X_\ell) \in E \Leftrightarrow X_k \not\perp\!\!\!\perp X_\ell | X_S$$

where $S = \{1, 2, ..., p\}/\{k, \ell\}$. Therefore:

$$\mathcal{N}(X_k) = \{\ell : \ell \subset \{1, 2, ..., p\}/\{k\}, X_k \not\perp\!\!\!\perp X_\ell | X_S \text{ where } S = \{1, 2, ..., p\}/\{k, \ell\}\}$$

For feature selection recall that:

$$\tilde{S} = \{j : j \subset \{1, 2, ..., p\}, Y \not\perp\!\!\!\perp X_j | X_{-j}\}$$

Neighborhood selection as feature selection cont.

For neighborhood selection of X_k in undirected graphical model:

$$\mathcal{N}(X_k) = \{\ell : \ell \subset \{1, 2, ..., p\}/\{k\}, X_k \not\perp\!\!\!\perp X_\ell | X_S \text{ where } S = \{1, 2, ..., p\}/\{k, \ell\}\}$$

For feature selection of Y recall that:

$$\tilde{S} = \{j : j \subset \{1, 2, ..., p\}, Y \not\perp\!\!\!\perp X_j | X_{-j}\}$$

Therefore:
Neighborhood selection for X_k is equivalent to feature selection where X_k represents Y and the remaining features $\{X_\ell : \ell \in \{1, 2, ..., p\}/\{k\}\}$ is feature set
And, to learn the full graph structure, run neighborhood selection for all p nodes $(X_1, X_2, ..., X_p)$.

Summarizing neighborhood selection for undirected graphical model learning

- For each node X_k, $k \in \{1, 2, ..., p\}$, neighbors (i.e. connected edges can be learned by performing feature selection where the feature set is all nodes excluding X_k
- Need to learn the neighbors for all p nodes to create entire graph
- Equivalent to solving p feature selection problems

Neighborhood selection code

```python
import numpy as np
from sklearn.linear_model import LassoCV
from sklearn.preprocessing import StandardScaler

def neighborhood_selection(X, alpha=0.5):
    n_samples, n_features = X.shape
    adjacency_matrix = np.zeros((n_features, n_features))

    # Standardize data
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    for j in range(n_features):
        # Response is X_j
        y = X_scaled[:, j]
        # Predictors are all other variables
        X_rest = np.delete(X_scaled, j, axis=1)

        # Fit Lasso with cross-validation if alpha is None
        if alpha is None:
            lasso = LassoCV(cv=5, positive=False).fit(X_rest, y)
            coef = lasso.coef_
        else:
            lasso = Lasso(alpha=alpha).fit(X_rest, y)
            coef = lasso.coef_

        # Fill adjacency matrix
        idx = 0
        for k in range(n_features):
            if k == j:
                continue
            if coef[idx] != 0:
                adjacency_matrix[j, k] = 1
            idx += 1

    # Symmetrize adjacency (OR rule)
    adjacency_matrix = np.maximum(adjacency_matrix, adjacency_matrix.T)
    return adjacency_matrix
```

Housing data example

For housing data recall features: X_1 : square footage; X_2 : Number of bedrooms; X_3 : Neighborhood quality score

```
df = pd.read_csv("Lecture_1_HousePrices.csv")
X = df.drop("price", axis=1)
adj = neighborhood_selection(X)
print("Estimated adjacency matrix:")
print(adj)
```

```
Estimated adjacency matrix:
[[0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 0.]]
```

After performing neighborhood selection, the graph adjacency matrix

$X_1 \not\perp X_2|X_3, X_1 \perp X_3|X_2, X_2 \perp X_3|X_1$

Interpreting undirected graphical model

- For each node X_k, $k \in \{1, 2, ..., p\}$, neighbors (i.e. connected edges can be learned by performing feature selection where the feature set is all nodes excluding X_k
- Need to learn the neighbors for all p nodes to create entire graph
- Equivalent to solving p feature selection problems